Scheduling Computation Graphs for Fun and Profit

Ryang Sohn

Pohang University of Science and Technology

May 22, 2026

Outline

Motivation

Overall strategy

1. First thought: “operator fusion is a well researched problem”

Motivation

Overall strategy

1. First thought: “operator fusion is a well researched problem”
   - How do production-grade optimizers approach this? (XLA, TensorRT, TVM, etc.)
   - They usually employ rule-based methods (e.g., epilogue-fuse ReLU after Matmul)

Motivation

Overall strategy

1. First thought: "operator fusion is a well researched problem"
   - How do production-grade optimizers approach this? (XLA, TensorRT, TVM, etc.)
   - They usually employ rule-based methods (e.g., epilogue-fuse ReLU after Matmul)
2. Rule-based methods work well in practice
   - An operator's performance characteristics are often associated with its kind
   - e.g., ReLU is memory bound and (non-skinny) Matmul is compute bound

Motivation

Overall strategy

1. First thought: "operator fusion is a well researched problem"
   - How do production-grade optimizers approach this? (XLA, TensorRT, TVM, etc.)
   - They usually employ rule-based methods (e.g., epilogue-fuse ReLU after Matmul)
2. Rule-based methods work well in practice
   - An operator's performance characteristics are often associated with its kind
   - e.g., ReLU is memory bound and (non-skinny) Matmul is compute bound
3. However, in this challenge...
   - There are only two kinds of operator (Matmul and Pointwise)
   - Each operator can have arbitrary cost using `base_cost`
   - Fusion opportunities are unlimited

Motivation

Overall strategy

1. First thought: "operator fusion is a well researched problem"
   - How do production-grade optimizers approach this? (XLA, TensorRT, TVM, etc.)
   - They usually employ rule-based methods (e.g., epilogue-fuse ReLU after Matmul)
2. Rule-based methods work well in practice
   - An operator's performance characteristics are often associated with its kind
   - e.g., ReLU is memory bound and (non-skinny) Matmul is compute bound
3. However, in this challenge...
   - There are only two kinds of operator (Matmul and Pointwise)
   - Each operator can have arbitrary cost using `base_cost`
   - Fusion opportunities are unlimited*

Motivation

Overall strategy

1. First thought: "operator fusion is a well researched problem"
   - How do production-grade optimizers approach this? (XLA, TensorRT, TVM, etc.)
   - They usually employ rule-based methods (e.g., epilogue-fuse ReLU after Matmul)
2. Rule-based methods work well in practice
   - An operator's performance characteristics are often associated with its kind
   - e.g., ReLU is memory bound and (non-skinny) Matmul is compute bound
3. However, in this challenge...
   - There are only two kinds of operator (Matmul and Pointwise)
   - Each operator can have arbitrary cost using `base_cost`
   - Fusion opportunities are unlimited*

Search-based method is needed for this challenge

Overview

- The solver needs to determine:
  1. Operators to fuse
  2. Retention of output tensors
  3. Iteration order (snake ordering is optimal)
  4. Tile size

Overview

- The solver needs to determine:
  1. Operators to fuse
  2. Retention of output tensors
  3. Iteration order (snake ordering is optimal)
  4. Tile size
- Retained tensor's lifetime is limited and fusion constraints are relaxed

Overview

- The solver needs to determine:
  1. Operators to fuse
  2. Retention of output tensors
  3. Iteration order (snake ordering is optimal)
  4. Tile size
- Retained tensor's lifetime is limited and fusion constraints are relaxed

Our overall strategy

Retention is local, fusion is global; fuse the nodes first, then consider retention.

Fusion Strategy

- Recomputation of operators means larger search space
  - How do we tackle the huge search space?

Fusion Strategy

- Recomputation of operators means larger search space
  - How do we tackle the huge search space?
- Hu et al., Optimal Kernel Orchestration for Tensor Programs with Korch. (ASPLOS '24)
  1. Enumerate convex subgraphs of the given graph
  2. Benchmark the subgraphs to get latency
  3. Formulate subgraph selection as BLP (Binary Linear Programming)
  4. Solve the BLP to get optimal fusion strategy

Fusion Strategy

- Recomputation of operators means larger search space
  - How do we tackle the huge search space?
- Hu et al., Optimal Kernel Orchestration for Tensor Programs with Korch. (ASPLOS '24)
  1. Enumerate convex subgraphs of the given graph
  2. Benchmark the subgraphs to get latency
  3. Formulate subgraph selection as BLP (Binary Linear Programming)
  4. Solve the BLP to get optimal fusion strategy
- In practice, we had to compromise
  - Cap the maximum size for enumerated convex subgraphs
  - Limit the BLP solver's running time

Retention Strategy

Partitioning in disguise

- Retained tensor's producer and consumer should be in separate subgraphs
- In other words, selecting retained tensors means partitioning a subgraph
- How to partition efficiently?

Retention Strategy

Partitioning in disguise

- Retained tensor's producer and consumer should be in separate subgraphs
- In other words, selecting retained tensors means partitioning a subgraph
- How to partition efficiently?
- Shi et al., Welder: Scheduling Deep Learning Memory Access via Tile-graph. (OSDI '23)
  - Introduces `GraphConnecting` algorithm for assigning memory tiers for tensors

Retention Strategy

Partitioning in disguise

- Retained tensor's producer and consumer should be in separate subgraphs
- In other words, selecting retained tensors means partitioning a subgraph
- How to partition efficiently?
- Shi et al., Welder: Scheduling Deep Learning Memory Access via Tile-graph. (OSDI '23)
  - Introduces `GraphConnecting` algorithm for assigning memory tiers for tensors
- Our partitioner selects ephemeral or fast memory for tensors
  - Iterate over each internal tensor and determine whether the tensor should be retained
  - Based on `GraphConnecting`, we extended it to use beam search for better results

And that's a Wrap!

For questions, leave an issue in our GitHub repository:
`https://github.com/sohnryang/mlsys2026-graph-scheduling`